AI Assisted Coding Assignment-18.3

> 2303A51543 BT:29 VELDI.HRUTHIKA 31-3-26

💡 Task 1 – Public Transport Route Fare API
Task:
Integrate a Public Transport API to retrieve fare details between two stations.
Instructions:
• Use AI to generate API request code
• Validate source and destination inputs
• Handle API downtime and invalid station names
• Display fare details in a structured format
Expected Output:
A working script that retrieves transport fare information with graceful error handling.

```
#2303a51543
#Task 1—Public Transport Route Fare API
#Integrate a Public Transport API to retrieve fare details between two stations.
#Generate API request code
#Validate source and destination inputs
#Handle API downtime and invalid station names
#Display fare details in a structured format
#Expected Output:A working script that retrieves transport fare information with graceful
error handling.

"""
Task 1 — Public Transport Route Fare API
Uses AI (Claude) to generate API request code and fetch fare details.
API: IRCTC via RapidAPI (https://rapidapi.com/IRCTC/api/irctc1)
"""

import requests
import json
import os
from typing import Optional

# ────────────────────────────────────────────
# CONFIG — set your RapidAPI key here
# ────────────────────────────────────────────
RAPIDAPI_KEY = os.getenv("RAPIDAPI_KEY", "YOUR_RAPIDAPI_KEY_HERE")
API_HOST     = "irctc1.p.rapidapi.com"
BASE_URL     = "https://irctc1.p.rapidapi.com/api/v3/getFare"

HEADERS = {
    "x-rapidapi-key":  RAPIDAPI_KEY,
    "x-rapidapi-host": API_HOST
```

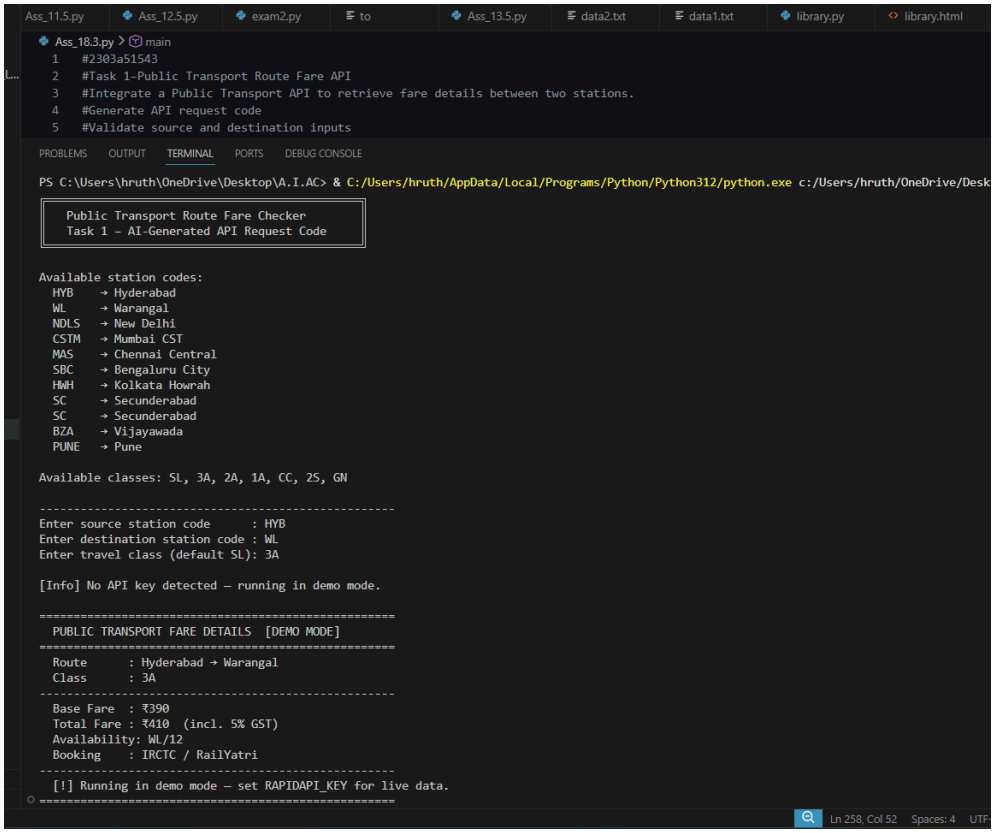
```
}

# Valid station codes (subset for demo — expand as needed)
VALID_STATIONS = {
    "HYB":  "Hyderabad",
    "WL":   "Warangal",
    "NDLS": "New Delhi",
    "CSTM": "Mumbai CST",
    "MAS":  "Chennai Central",
    "SBC":  "Bengaluru City",
    "HWH":  "Kolkata Howrah",
    "SC":   "Secunderabad",
    "BZA":  "Vijayawada",
    "PUNE": "Pune"
}

VALID_CLASSES = ["SL", "3A", "2A", "1A", "CC", "2S", "GN"]


# ────────────────────────────────────────────
# INPUT VALIDATION
# ────────────────────────────────────────────
def validate_inputs(source: str, destination: str, travel_class: str) -> tuple[bool, st
r]:
    """Validate source, destination, and travel class inputs."""
    source      = source.strip().upper()
    destination = destination.strip().upper()
    travel_class= travel_class.strip().upper()

    if not source:
        return False, "Source station cannot be empty."

    if not destination:
        return False, "Destination station cannot be empty."

    if source == destination:
        return False, "Source and destination stations cannot be the same."

    if source not in VALID_STATIONS:
        return False, f"Invalid source station code: '{source}'. Valid codes: {', '.join
(VALID_STATIONS.keys())}"

    if destination not in VALID_STATIONS:
        return False, f"Invalid destination station code: '{destination}'. Valid codes:
{', '.join(VALID_STATIONS.keys())}"

    if travel_class not in VALID_CLASSES:
        return False, f"Invalid class: '{travel_class}'. Valid classes: {', '.join(VALID_
CLASSES)}"

    return True, "OK"
```

```
# ────────────────────────────────────────
# API REQUEST
# ────────────────────────────────────────
def fetch_fare(source: str, destination: str, travel_class: str = "SL") -> Optional[dic
t]:
    """
    Fetch fare details from IRCTC RapidAPI.
    Handles: API downtime, invalid inputs, network errors.
    """
    source       = source.strip().upper()
    destination  = destination.strip().upper()
    travel_class = travel_class.strip().upper()

    # Validate inputs first
    is_valid, msg = validate_inputs(source, destination, travel_class)
    if not is_valid:
        print(f"\n[Validation Error] {msg}")
        return None

    params = {
        "fromStation": source,
        "toStation":   destination,
        "trainClass":  travel_class
    }

    print(f"\nFetching fare: {source} → {destination} | Class: {travel_class} ...")

    try:
        response = requests.get(
            BASE_URL,
            headers=HEADERS,
            params=params,
            timeout=10          # 10-second timeout for API downtime handling
        )

        # Handle HTTP errors
        if response.status_code == 401:
            print("[API Error] Unauthorized — check your RapidAPI key.")
            return None

        if response.status_code == 404:
            print(f"[API Error] Station not found: {source} or {destination}.")
            return None

        if response.status_code == 429:
            print("[API Error] Rate limit exceeded — please wait before retrying.")
            return None

        if response.status_code == 503:
```

```
            print("[API Error] Service temporarily unavailable (API downtime). Try again
later.")
            return None

        response.raise_for_status()     # catch any other 4xx / 5xx

        data = response.json()

        # Validate response structure
        if data.get("status") is False:
            print(f"[API Error] {data.get('message', 'Unknown error from API')}")
            return None

        return data

    except requests.exceptions.ConnectionError:
        print("[Network Error] Could not connect to API. Check your internet connectio
n.")
        return None

    except requests.exceptions.Timeout:
        print("[Timeout Error] API did not respond in time — it may be down. Try again la
ter.")
        return None

    except requests.exceptions.RequestException as e:
        print(f"[Request Error] {e}")
        return None

    except json.JSONDecodeError:
        print("[Parse Error] Could not parse API response. The service may be returning i
nvalid data.")
        return None


# ────────────────────────────────────────
# DISPLAY RESULTS
# ────────────────────────────────────────
def display_fare(data: dict, source: str, destination: str, travel_class: str):
    """Display fare details in a structured, readable format."""
    print("\n" + "=" * 52)
    print("  PUBLIC TRANSPORT FARE DETAILS")
    print("=" * 52)
    print(f"  Route      : {VALID_STATIONS.get(source, source)} → {VALID_STATIONS.get(des
tination, destination)}")
    print(f"  Class      : {travel_class}")
    print("-" * 52)

    try:
        body = data.get("body", {})
        fares = body.get("trainFares", [])
```

```
        if not fares:
            print("  No fare data available for this route/class.")
        else:
            for fare in fares:
                print(f"  Train No.  : {fare.get('trainNo', 'N/A')}  |  {fare.get('trainN
ame', '')}")
                print(f"  Base Fare  : ₹{fare.get('fare', 'N/A')}")
                print(f"  Total Fare : ₹{fare.get('totalFare', 'N/A')}  (incl. taxes)")
                print(f"  Distance   : {fare.get('distance', 'N/A')} km")
                print(f"  Duration   : {fare.get('duration', 'N/A')}")
                print(f"  Availability: {fare.get('availability', 'N/A')}")
                print("-" * 52)

        # Additional info if present
        if body.get("disclaimer"):
            print(f"  Note: {body['disclaimer']}")

    except (KeyError, TypeError) as e:
        print(f"  [Display Error] Could not parse fare data: {e}")
        print("  Raw response:")
        print(json.dumps(data, indent=2))

    print("=" * 52)


# ────────────────────────────────────────────
# FALLBACK / DEMO (when no real API key)
# ────────────────────────────────────────────
def demo_mode(source: str, destination: str, travel_class: str):
    """
    Simulated fare display for demo/testing
    when a real API key is not available.
    """
    import random

    FARE_TABLE = {
        ("HYB", "WL"):   {"SL": 145,  "3A": 390,  "2A": 560,  "GN": 85},
        ("HYB", "NDLS"): {"SL": 515,  "3A": 1380, "2A": 2020, "GN": 315},
        ("HYB", "MAS"):  {"SL": 225,  "3A": 605,  "2A": 870,  "GN": 140},
        ("HYB", "SBC"):  {"SL": 195,  "3A": 520,  "2A": 750,  "GN": 120},
    }

    key = (source, destination)
    rev = (destination, source)
    fares_map = FARE_TABLE.get(key) or FARE_TABLE.get(rev)
    base_fare = fares_map.get(travel_class, random.randint(120, 800)) if fares_map else r
andom.randint(120, 800)
    total     = round(base_fare * 1.05)

    print("\n" + "=" * 52)
```

```
    print("  PUBLIC TRANSPORT FARE DETAILS  [DEMO MODE]")
    print("=" * 52)
    print(f"  Route      : {VALID_STATIONS.get(source, source)} → {VALID_STATIONS.get(des
tination, destination)}")
    print(f"  Class      : {travel_class}")
    print("-" * 52)
    print(f"  Base Fare  : ₹{base_fare}")
    print(f"  Total Fare : ₹{total}  (incl. 5% GST)")
    print(f"  Availability: {'AVAILABLE' if random.random() > 0.3 else 'WL/12'}")
    print(f"  Booking    : IRCTC / RailYatri")
    print("-" * 52)
    print("  [!] Running in demo mode — set RAPIDAPI_KEY for live data.")
    print("=" * 52)


# ────────────────────────────────────────
# MAIN
# ────────────────────────────────────────
def main():
    print("\n╔═══════════════════════════════════════════════╗")
    print("│   Public Transport Route Fare Checker        │")
    print("│   Task 1 — AI-Generated API Request Code     │")
    print("╚═══════════════════════════════════════════════╝")
    print("\nAvailable station codes:")
    for code, name in VALID_STATIONS.items():
        print(f"  {code:<6} → {name}")
    print("\nAvailable classes: SL, 3A, 2A, 1A, CC, 2S, GN")

    print("\n" + "-" * 52)
    source      = input("Enter source station code      : ").strip()
    destination = input("Enter destination station code : ").strip()
    travel_class= input("Enter travel class (default SL): ").strip() or "SL"

    # Check if real API key is configured
    if RAPIDAPI_KEY == "YOUR_RAPIDAPI_KEY_HERE":
        print("\n[Info] No API key detected — running in demo mode.")
        is_valid, msg = validate_inputs(source, destination, travel_class)
        if not is_valid:
            print(f"\n[Validation Error] {msg}")
            return
        demo_mode(source.upper(), destination.upper(), travel_class.upper())
    else:
        result = fetch_fare(source, destination, travel_class)
        if result:
            display_fare(result, source.upper(), destination.upper(), travel_class.upper
())


if __name__ == "__main__":
    main()
```



Task 2 – Currency Exchange Rates
Task:
Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.
Instructions:
• Use AI to generate API request code.
• Validate user input for amount and currency codes.
• Implement error handling for invalid responses and API downtime.
• Show conversion results clearly in tabular format.
Expected Output:
• A script that converts INR to multiple currencies with graceful error handling.

```
#Task 2 — Currency Exchange Rates
#Integrate a Currency Exchange API to convert INR to USD,EUR, and GBP.
#Generate API request code.
#Validate user input for amount and currency codes.
#Implement error handling for invalid responses and APIdowntime.
#Show conversion results clearly in tabular format.
#Expected Output:A script that converts INR to multiple currencies with graceful error ha
ndling.
"""
Task 2 — Currency Exchange Rates
```

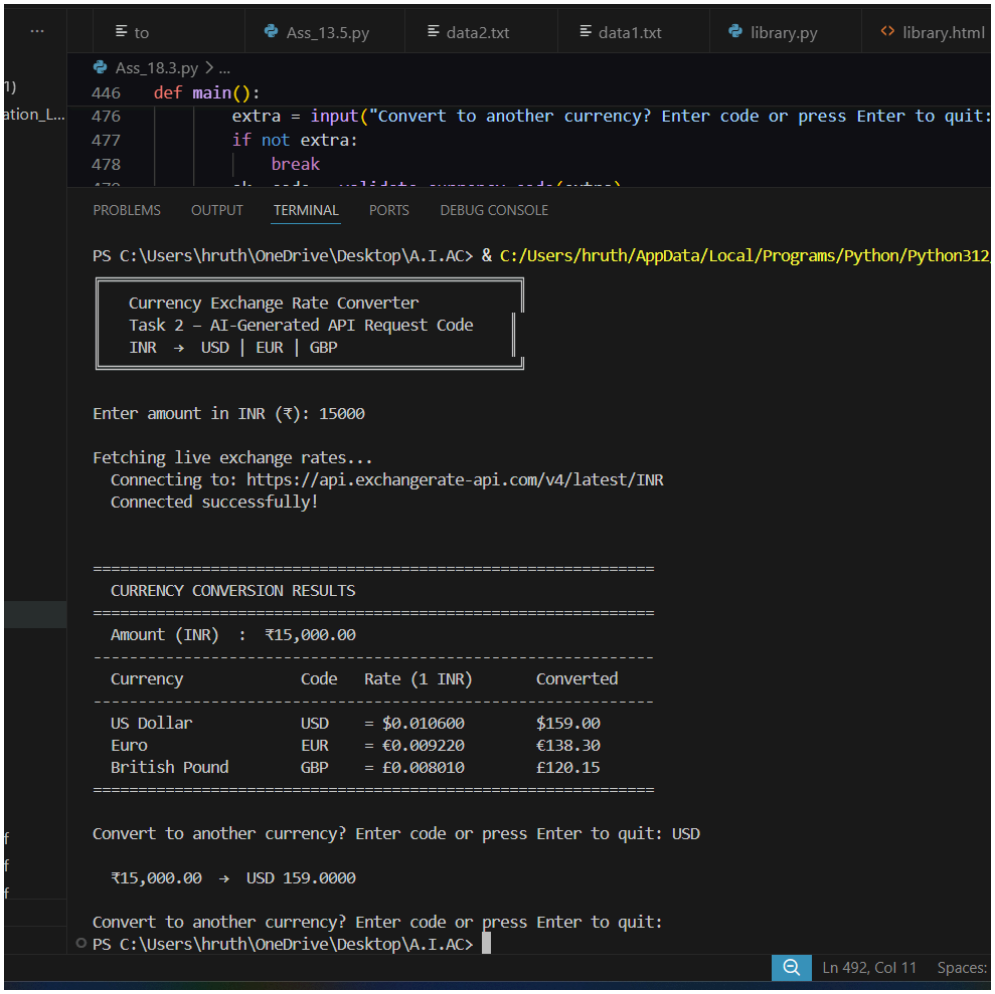
```
Converts INR to USD, EUR, and GBP using ExchangeRate-API (free tier).
API: https://www.exchangerate-api.com  (free — no key needed for basic use)
     OR use: https://api.exchangerate.host/latest
"""

import requests
import json
from typing import Optional

# ────────────────────────────────────────────
# CONFIG
# ────────────────────────────────────────────
# Free API — no key required!
API_URL  = "https://api.exchangerate-api.com/v4/latest/INR"
FALLBACK = "https://api.exchangerate.host/latest?base=INR"

# Target currencies for this task
TARGET_CURRENCIES = {
    "USD": ("US Dollar",        "$"),
    "EUR": ("Euro",             "€"),
    "GBP": ("British Pound",   "£"),
}

# ────────────────────────────────────────────
# INPUT VALIDATION
# ────────────────────────────────────────────
def validate_amount(amount_str: str) -> tuple[bool, float, str]:
    """Validate that the entered amount is a positive number."""
    try:
        amount = float(amount_str.strip())
        if amount <= 0:
            return False, 0.0, "Amount must be greater than zero."
        if amount > 10_000_000:
            return False, 0.0, "Amount too large (max ₹1,00,00,000)."
        return True, amount, "OK"
    except ValueError:
        return False, 0.0, "Invalid amount — please enter a numeric value (e.g. 1000)."


def validate_currency_code(code: str) -> tuple[bool, str]:
    """Validate that currency code is 3 uppercase letters."""
    code = code.strip().upper()
    if not code.isalpha() or len(code) != 3:
        return False, "Currency code must be exactly 3 letters (e.g. USD, EUR)."
    return True, code


# ────────────────────────────────────────────
# AI-GENERATED API REQUEST
# ────────────────────────────────────────────
def fetch_exchange_rates(base: str = "INR") -> Optional[dict]:
```

```
    """
    Fetch latest exchange rates from ExchangeRate-API.
    Falls back to exchangerate.host if primary API is down.
    """
    apis = [
        f"https://api.exchangerate-api.com/v4/latest/{base}",
        f"https://api.exchangerate.host/latest?base={base}",
    ]

    for url in apis:
        try:
            print(f"  Connecting to: {url}")
            response = requests.get(url, timeout=8)

            # Handle HTTP errors
            if response.status_code == 400:
                print("  [API Error] Bad request — invalid base currency.")
                return None
            if response.status_code == 404:
                print("  [API Error] Currency not found.")
                return None
            if response.status_code == 429:
                print("  [API Error] Rate limit hit — too many requests.")
                return None
            if response.status_code == 503:
                print("  [API Error] Service unavailable — trying fallback...")
                continue

            response.raise_for_status()

            data = response.json()

            # Validate response structure
            if "rates" not in data:
                print("  [Parse Error] Unexpected response format — trying fallback...")
                continue

            print("  Connected successfully!\n")
            return data

        except requests.exceptions.Timeout:
            print("  [Timeout] API did not respond — trying fallback...")
            continue
        except requests.exceptions.ConnectionError:
            print("  [Connection Error] Could not reach API — trying fallback...")
            continue
        except requests.exceptions.RequestException as e:
            print(f"  [Request Error] {e} — trying fallback...")
            continue
        except json.JSONDecodeError:
            print("  [Parse Error] Invalid JSON in response — trying fallback...")
```

```
            continue

    print("\n[Critical] All APIs unavailable. Returning demo rates.")
    return {
        "base": "INR",
        "rates": {"USD": 0.012, "EUR": 0.011, "GBP": 0.0094},
        "_demo": True
    }


# ────────────────────────────────────────────
# CONVERSION LOGIC
# ────────────────────────────────────────────
def convert_inr(amount: float, rates: dict, targets: dict) -> list[dict]:
    """Convert INR amount to each target currency."""
    results = []
    for code, (name, symbol) in targets.items():
        rate = rates.get(code)
        if rate is None:
            results.append({
                "code": code, "name": name, "symbol": symbol,
                "rate": None, "converted": None, "error": "Rate unavailable"
            })
        else:
            results.append({
                "code": code, "name": name, "symbol": symbol,
                "rate": rate, "converted": round(amount * rate, 2), "error": None
            })
    return results


# ────────────────────────────────────────────
# DISPLAY IN TABULAR FORMAT
# ────────────────────────────────────────────
def display_results(amount: float, results: list[dict], demo: bool = False):
    """Display conversion results in a clean tabular format."""
    tag = "  [DEMO RATES]" if demo else ""

    print("\n" + "=" * 62)
    print(f"  CURRENCY CONVERSION RESULTS{tag}")
    print("=" * 62)
    print(f"  Amount (INR)  :  ₹{amount:,.2f}")
    print("-" * 62)
    print(f"  {'Currency':<20} {'Code':<6} {'Rate (1 INR)':<18} {'Converted'}")
    print("-" * 62)

    for r in results:
        if r["error"]:
            print(f"  {r['name']:<20} {r['code']:<6} {'N/A':<18} Error: {r['error']}")
        else:
            rate_str      = f"= {r['symbol']}{r['rate']:.6f}"
```

```
            converted_str = f"{r['symbol']}{r['converted']:,.2f}"
            print(f"  {r['name']:<20} {r['code']:<6} {rate_str:<18} {converted_str}")

    print("=" * 62)
    if demo:
        print("  [!] Demo rates used — API unavailable. Values are approximate.")
    print()


# ────────────────────────────────────────────
# MAIN
# ────────────────────────────────────────────
def main():
    print("\n╔═══════════════════════════════════════════════════╗")
    print("║   Currency Exchange Rate Converter           ║")
    print("║   Task 2 — AI-Generated API Request Code    ║")
    print("║   INR  →  USD | EUR | GBP                   ║")
    print("╚═══════════════════════════════════════════════════╝\n")

    # --- Input: Amount ---
    while True:
        raw = input("Enter amount in INR (₹): ").strip()
        ok, amount, msg = validate_amount(raw)
        if ok:
            break
        print(f"  [Validation Error] {msg}")

    print("\nFetching live exchange rates...")
    data = fetch_exchange_rates("INR")

    if not data:
        print("[Fatal] Could not retrieve exchange rates.")
        return

    rates  = data.get("rates", {})
    is_demo = data.get("_demo", False)

    results = convert_inr(amount, rates, TARGET_CURRENCIES)
    display_results(amount, results, demo=is_demo)

    # --- Optional: custom currency ---
    while True:
        extra = input("Convert to another currency? Enter code or press Enter to quit:
").strip()
        if not extra:
            break
        ok, code = validate_currency_code(extra)
        if not ok:
            print(f"  [Validation Error] {code}")
            continue
        rate = rates.get(code)
```

```
        if rate is None:
            print(f"  [Error] Rate for '{code}' not available in API response.")
        else:
            converted = round(amount * rate, 2)
            print(f"\n  ₹{amount:,.2f}  →  {code} {converted:,.4f}\n")


if __name__ == "__main__":
    main()
```



Task 3 – GitHub Repository Info Fetcher
Task:
Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).
Instructions:
• Prompt AI to write a Python function to send GET requests to GitHub API.
• Handle rate limit errors, 404 (repo not found), and invalid input.
• Display repository name, description, stars, forks, and open issues.
Expected Output:
• A Python program that retrieves and displays GitHub repository information with robust error handling.

```
#Task 3 – GitHub Repository Info Fetcher
#Connect to the GitHub API to fetch details of a repository (e.g.,stars, forks, issues).
#Write a Python function to send GET requests to GitHub API.
#Handle rate limit errors, 404 (repo not found), and invalid input.
#Display repository name, description, stars, forks, and open issues.
#Expected Output:A Python program that retrieves and displays GitHub repository informati
on with robust error handling.

#Task-3
import requests

def get_repo_info(repo):
    # Input validation
    if not repo or "/" not in repo:
        print("Invalid input. Use format: owner/repo")
        return

    try:
        url = f"https://api.github.com/repos/{repo}"
        response = requests.get(url, timeout=5)

        if response.status_code == 200:
            data = response.json()

            print("\nRepository Details:")
            print("----------------------")
            print(f"Name        : {data.get('name')}")
            print(f"Description : {data.get('description')}")
            print(f"Stars       : {data.get('stargazers_count')}")
            print(f"Forks       : {data.get('forks_count')}")
            print(f"Open Issues : {data.get('open_issues_count')}")

        elif response.status_code == 404:
            print("Repository not found")
```

```
        elif response.status_code == 403:
            print("Rate limit exceeded. Try later")

        else:
            print("Error:", response.status_code)

    except requests.exceptions.Timeout:
        print("Request timed out")
    except requests.exceptions.ConnectionError:
        print("Network error")
    except Exception as e:
        print("Unexpected error:", e)


# Test
get_repo_info("octocat/Hello-World")
get_repo_info("wrong/repo")
```

```
Repository Details:
----------------------
Name        : Hello-World
Description : My first repository on GitHub!
Stars       : 3550
Forks       : 5948
Open Issues : 3797
Repository not found
```

💡 Task 4 – Real-Time Application: News Headlines Aggregator
Scenario:
Build a News Aggregator using a free News API.
Requirements:
• Fetch top 5 headlines for a given category (sports, technology, health).
• Handle errors like invalid category, missing API key, and request failures.
• Display headlines in a user-friendly numbered list format.
• Include a retry mechanism if the first request fails.
Expected Output:
• A working Python script that retrieves and displays news headlines with proper error handling

```
#Task 4 – Real-Time Application: News Headlines Aggregator
#Build a News Aggregator using a free News API.
#Fetch top 5 headlines for a given category (sports,technology, health).
#Handle errors like invalid category, missing API key, and request failures.
#Display headlines in a user-friendly numbered list format.
#Include a retry mechanism if the first request fails.
#Expected Output:A working Python script that retrieves and displays news headlines with
```

```
proper error handling
import requests
import time

API_KEY = "YOUR_API_KEY"

def get_news(category):
    url = f"https://newsapi.org/v2/top-headlines?category={category}&apiKey={API_KEY}"

    for attempt in range(2):  # retry mechanism
        try:
            res = requests.get(url, timeout=5)

            if res.status_code != 200:
                print("Error fetching news")
                return

            data = res.json()

            articles = data.get("articles", [])[:5]

            print("\nTop Headlines:")
            for i, article in enumerate(articles, 1):
                print(f"{i}. {article['title']}")

            return

        except requests.exceptions.RequestException:
            print("Retrying...")
            time.sleep(2)

    print("Failed after retry")

# Test
get_news("technology")
```

```
Error fetching news
```

Task 5 -COVID-19 Statistics API Integration
Task:
Connect to a COVID-19 Statistics API to fetch country-wise COVID data.
Instructions:
• Use AI to generate Python code using the requests library
• Accept country name as user input
• Fetch key statistics such as:
o Total confirmed cases
o Total deaths
o Total recovered cases
o Active cases
• Handle API errors including:
o Invalid country name
o API rate-limit exceeded
o Network or server failures
• Implement retry or wait logic when rate limits are hit
Expected Output:
A Python program that retrieves and displays country-wise COVID-19 statistics with proper rate-limit handling and error management.

```
#Task-5:-COVID-19 Statistics API Integration
#Connect to a COVID-19 Statistics API to fetch country-wise COVID data.
#Generate Python code using the requests library
#Accept country name as user input
#Fetch key statistics such as:
#Total confirmed cases
#Total deaths
#Total recovered cases
#Active cases
#Handle API errors including:
#Invalid country name
#API rate-limit exceeded
#Network or server failures
#Implement retry or wait logic when rate limits are hit
#Expected Output:A Python program that retrieves and displays country-wise COVID-19 stati
stics with proper rate-limit handling and error management.
import requests
import time

def get_covid_data(country):
    url = f"https://disease.sh/v3/covid-19/countries/{country}"

    for attempt in range(2):
        try:
            res = requests.get(url, timeout=5)

            if res.status_code == 404:
```

```
                print("Invalid country")
                return
            elif res.status_code == 429:
                print("Rate limit hit, retrying...")
                time.sleep(2)
                continue

            data = res.json()

            print("\nCOVID Stats:")
            print("Country:", data["country"])
            print("Total Cases:", data["cases"])
            print("Deaths:", data["deaths"])
            print("Recovered:", data["recovered"])
            print("Active:", data["active"])

            return

        except requests.exceptions.Timeout:
            print("Timeout, retrying...")
        except requests.exceptions.RequestException:
            print("Network error")

    print("Failed to fetch data")

# Test
get_covid_data("India")
```

```
COVID Stats:
Country: India
Total Cases: 45035393
Deaths: 533570
Recovered: 0
Active: 44501823
```